False Sharing and Scalability in Multicore Processors: Impact of Cache-Line Data Layout on Parallel Performance

Walter Alfaro, Allan Arrieta

***Abstract*—Modern multicore processors keep private caches coherent at the granularity of a cache line, typically 64 bytes, rather than individual variables. When threads running on different cores write to independent variables that happen to reside in the same cache line, the coherence protocol forces the line to migrate repeatedly between cores, even though no data is logically shared. This phenomenon, known as false sharing, generates invalidation traffic and cache-to-cache transfers that can severely limit the scalability of parallel programs and is a recurring source of performance regressions in large systems such as the Linux kernel. This paper investigates how the distribution of variables across cache lines affects the performance of a parallel program on a hybrid x86-64 Intel Core i7-1260P (Alder Lake) processor, which combines performance and efficiency cores. We study a microbenchmark in which each thread, pinned to a dedicated core, updates its own independent counter, comparing a layout where the counters are stored contiguously against layouts in which they are separated through alignment and padding, as well as per-thread accumulation. The analysis relates the observed execution time and scalability with the number of threads to the behavior of the MESI-family coherence protocol, and uses hardware performance counters and the Linux `perf c2c` tool to identify contended cache lines through HITM (load hit in a modified line) events. In addition, the memory and cache-capacity overhead introduced by padding is quantified, together with the mitigation strategies documented by the Linux kernel community, in order to characterize the trade-off between coherence-traffic reduction and memory footprint and to derive practical guidelines for cache-aware data layout.**

***Index Terms*—False sharing, cache coherence, MESI, multicore processors, cache line, padding, performance analysis, perf c2c, Linux kernel.**

REFERENCES

[1] F. Tang, "False sharing," The Linux Kernel Documentation, Kernel Hacking Guides, 2023, accessed: Nov. 5, 2026. [Online]. Available: https://kernel.org/doc/html/latest/kernel-hacking/false-sharing.html

[2] J. Mario, "C2C – false sharing detection in Linux perf," Sep. 2016, accessed: Nov. 5, 2026. [Online]. Available: https://joemario.github.io/blog/2016/09/01/c2c-blog/

[3] ——, "perf-c2c-usage-files: Example reports and false sharing test program," GitHub repository, accessed: Nov. 5, 2026. [Online]. Available: https://github.com/joemario/perf-c2c-usage-files

[4] Linux perf developers, "perf-c2c(1) – shared data C2C/HITM analyzer," Linux manual page, accessed: Nov. 5, 2026. [Online]. Available: https://man7.org/linux/man-pages/man1/perf-c2c.1.html

[5] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual, Volume 1*, May 2024, rev. 051. Accessed: Nov. 5, 2026. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/814198/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html

[6] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., Nov. 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[7] J. Torrellas, M. S. Lam, and J. L. Hennessy, "False sharing and spatial locality in multiprocessor caches," *IEEE Transactions on Computers*, vol. 43, no. 6, pp. 651–663, Jun. 1994.

[8] T. Liu and E. D. Berger, "SHERIFF: Precise detection and automatic mitigation of false sharing," in *Proc. ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, Portland, OR, USA, 2011, pp. 3–18.

[9] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[10] V. Nagarajan, D. J. Sorin, M. D. Hill, and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., ser. Synthesis Lectures on Computer Architecture. Springer, 2020.